JAVA DSA • COMPLETE INTERVIEW SERIES

TREES & BINARY TREES

FUNDAMENTALS TO DEEP — MASTER HANDBOOK

Concept clarity • diagrams in text • algorithm intuition • complete Java methods • dry runs • complexity proofs • edge cases • interview patterns • practice problems

Coverage map
Tree foundations and mathematical properties; binary tree types; BST operations; DFS/BFS; recursive and iterative traversals; height/depth/diameter/width; balance; LCA; path problems; boundary/zigzag/views; serialization; reconstruction from traversals; Morris traversal; flattening; symmetry; validation; kth smallest; successor/predecessor; interview problem bank.

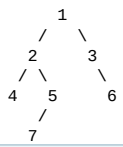
Conventions used throughout
Java 17 syntax. Tree height is measured in **nodes**: empty tree = 0, leaf = 1. Depth is measured in **edges**: root depth = 0. Diameter is measured in **edges**. Unless stated, BST keys are distinct. These conventions matter because some platforms define height in edges.

Study order: understand the invariant → trace a small tree manually → write the method → test empty, one-node and skewed trees → state time and auxiliary space.

How to study this handbook

Stage	What you must be able to do before moving on
1. Foundations	Explain root, parent, leaf, height, depth, subtree, full/complete/perfect; calculate basic formulas.
2. Traversals	Write recursive and iterative preorder, inorder, postorder and BFS without looking.
3. Tree DP	Derive height, diameter and balance using left/right return values.
4. Relationships	Solve LCA and path problems while stating assumptions about missing nodes and path endpoints.
5. Structural patterns	Implement views, boundary, zigzag, serialization and traversal reconstruction.
6. Advanced	Explain Morris threading, O(1) auxiliary space and why temporary links are restored.
7. Interview readiness	Dry-run edge cases, justify complexity and compare alternative approaches.

One sample tree used for dry runs



Nodes = 7; edges = 6; leaves = 4, 7, 6; height in nodes = 4; depth of node 7 = 3; width by actual-node count = 3. Root-to-leaf paths: 1-2-4, 1-2-5-7, 1-3-6.

Reusable imports and node class

```
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

On LeetCode-style platforms, TreeNode is often already provided. Do not redeclare it there. Place methods in the expected Solution class, and adapt method signatures to the platform.

1. Tree fundamentals and terminology

A tree is a connected, acyclic graph organized hierarchically. A rooted tree chooses one node as root. Every non-root node has exactly one parent. In a binary tree, each node has at most two child links: left and right. The position matters: left-child and right-child are not interchangeable.

Word	Exact meaning
Node	An element containing data and links/references to children.
Root	Topmost node; no parent.
Edge	A parent-child connection.
Parent / child	Direct predecessor / successor in the hierarchy.
Siblings	Nodes sharing the same parent.
Leaf	Node with zero children.
Internal node	Node with at least one child.
Ancestor	Any node on the path from root to a given node.
Descendant	Any node below a given node.
Subtree	A node plus all descendants connected below it.
Degree	Number of children; 0, 1 or 2 for a binary tree.
Depth	Number of edges from root; root depth = 0.
Height of node	Longest downward path to a leaf, using a stated edge/node convention.
Level	Often depth + 1 when root is level 1; some texts start root at level 0.

Tree properties and derivations

• A tree with n nodes has exactly n−1 edges. Proof idea: every node except root has one parent edge, so there are n−1 parent links.

• There is exactly one simple path between any two nodes. Two different simple paths would create a cycle.

• At level d (root at level 0), a binary tree can contain at most 2^d nodes.

• If height h is counted in levels/nodes, maximum nodes = 1+2+4+…+2^(h−1) = 2^h−1.

• A skewed tree of height h has h nodes. Thus for n nodes, height ranges from ceil(log2(n+1)) to n.

• For a full binary tree, every node has 0 or 2 children; number of leaves L = internal nodes I + 1.

• Perfect binary tree with h levels has 2^h−1 nodes and 2^(h−1) leaves.

• Complete tree: all levels except perhaps last are full; last level is filled left to right.

Tree types — do not mix these definitions

Type	Rule	Typical property
Full / proper	Every node has either 0 or 2 children	No node has exactly one child
Complete	All levels full except possibly last; last fills left-to-right	Efficient array storage; heaps use this shape
Perfect	All internal nodes have 2 children and all leaves same depth	Exactly 2^h−1 nodes for h levels
Balanced	Height is kept logarithmic or subtree heights are constrained	Exact definition depends on structure
Skewed	Every internal node has only one child	Height can be n; behaves like a list

Representations

Pointer representation: each node stores value and left/right references. Works for arbitrary shapes; unused child links are null.

Array representation: ideal for complete trees. With 0-based index i: left=2i+1, right=2i+2, parent=floor((i−1)/2). With 1-based indexing: left=2i, right=2i+1.

```
// 0-based array example:
// index:  0  1  2  3  4  5  6
// value:  1  2  3  4  5  -  6
// node at i=1 has left=3 and right=4.
```

Array storage wastes space for sparse/skewed trees because child positions can grow rapidly while many positions are null. Pointer nodes avoid this issue.

2. Binary Tree vs Binary Search Tree

A binary tree only restricts the number of children. It does not impose an ordering on values. A Binary Search Tree (BST) adds an ordering invariant. For distinct keys: every key in left subtree < node key < every key in right subtree. The rule applies to the entire subtree, not just immediate children.

Feature	Binary Tree	BST
Ordering	No fixed ordering	Left subtree < root < right subtree
Search	O(n) worst case	O(h); O(log n) if balanced, O(n) if skewed
Inorder	Arbitrary order	Ascending sorted order
Use	General hierarchy, expression trees	Ordered search, predecessor/successor, ranges
Duplicates	Can freely occur	Need explicit policy

BST search

```
TreeNode searchBST(TreeNode root, int key) {
    TreeNode cur = root;
    while (cur != null) {
        if (key == cur.val) return cur;
        if (key < cur.val) cur = cur.left;
        else cur = cur.right;
    }
    return null;
}
```

At every step one entire subtree is discarded. Complexity O(h) time and O(1) auxiliary space. If the BST is balanced, h=O(log n); if skewed, h=O(n).

BST insertion

```
TreeNode insertBST(TreeNode root, int key) {
    if (root == null) return new TreeNode(key);
    if (key < root.val) root.left = insertBST(root.left, key);
    else if (key > root.val) root.right = insertBST(root.right, key);
    // This version ignores duplicate keys.
    return root;
}
```

BST deletion — three cases

• Case 1: target is a leaf. Return null to remove it.

• Case 2: target has one child. Return its non-null child to connect parent directly to child.

• Case 3: target has two children. Replace value with inorder successor (smallest in right subtree) or predecessor (largest in left subtree), then delete that replacement from its original position.

```
TreeNode minValueNode(TreeNode node) {
    while (node.left != null) node = node.left;
    return node;
}

TreeNode deleteBST(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.val) {
        root.left = deleteBST(root.left, key);
    } else if (key > root.val) {
        root.right = deleteBST(root.right, key);
    } else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        TreeNode successor = minValueNode(root.right);
        root.val = successor.val;
        root.right = deleteBST(root.right, successor.val);
    }
    return root;
}
```

All three BST operations are O(h). Deletion is commonly misunderstood: after copying the successor's value, the successor node must also be deleted from the right subtree or the value appears twice.

3. BST interview algorithms

Validate a BST — range/bounds method

Checking only parent-child comparisons is insufficient. A node deep in the left subtree must be smaller than every ancestor bound, not merely smaller than its direct parent. Pass an allowed open interval down the recursion.

```
boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

boolean valid(TreeNode node, long low, long high) {
    if (node == null) return true;
    if (node.val <= low || node.val >= high) return false;
    return valid(node.left, low, node.val) &&
           valid(node.right, node.val, high);
}
```

Time O(n), recursion O(h). long bounds safely handle Integer.MIN_VALUE and Integer.MAX_VALUE. This version assumes duplicates are forbidden.

Kth smallest in BST

Inorder visits BST values in ascending order. Count visited nodes and stop at k. Validate k according to the problem constraints.

```
int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    int count = 0;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) {
            st.push(cur);
            cur = cur.left;
        }
        cur = st.pop();
        if (++count == k) return cur.val;
        cur = cur.right;
    }
    throw new IllegalArgumentException("k exceeds node count");
}
```

Kth largest

Reverse inorder (Right → Root → Left) visits values descending. Use the same stack pattern, but walk right first.

```
int kthLargest(TreeNode root, int k) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    int count = 0;
    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.right; }
        cur = st.pop();
        if (++count == k) return cur.val;
        cur = cur.left;
    }
    throw new IllegalArgumentException("k exceeds node count");
}
```

Inorder successor and predecessor

Successor = smallest key strictly greater than target. If target has a right subtree, successor is its minimum. Otherwise, walk from root and remember the last ancestor for which target lies in its left subtree. Predecessor is symmetric.

```
TreeNode successorBST(TreeNode root, int key) {
    TreeNode succ = null, cur = root;
    while (cur != null) {
        if (key < cur.val) { succ = cur; cur = cur.left; }
        else cur = cur.right;
    }
    return succ;
}

TreeNode predecessorBST(TreeNode root, int key) {
    TreeNode pred = null, cur = root;
    while (cur != null) {
        if (key > cur.val) { pred = cur; cur = cur.right; }
        else cur = cur.left;
    }
    return pred;
}
```

BST range sum

```
int rangeSumBST(TreeNode root, int low, int high) {
    if (root == null) return 0;
    if (root.val < low) return rangeSumBST(root.right, low, high);
    if (root.val > high) return rangeSumBST(root.left, low, high);
    return root.val
        + rangeSumBST(root.left, low, high)
        + rangeSumBST(root.right, low, high);
}
```

Pruning is the key: if current value is below low, its entire left subtree is also too small; if above high, its right subtree is too large.

4. DFS traversal concepts

Depth-First Search explores one branch deeply before returning. The three classic DFS traversals differ only in when the current/root node is processed.

Name	Order	Mnemonic / use
Preorder	Root → Left → Right	Root first; copy/serialize tree, prefix notation
Inorder	Left → Root → Right	BST gives ascending order
Postorder	Left → Right → Root	Children first; height, diameter, deletion, tree DP

Recursive preorder

```
void preorder(TreeNode node, List<Integer> out) {
    if (node == null) return;   // base case
    out.add(node.val);          // process root
    preorder(node.left, out);
    preorder(node.right, out);
}
```

Recursive inorder

```
void inorder(TreeNode node, List<Integer> out) {
    if (node == null) return;
    inorder(node.left, out);
    out.add(node.val);
    inorder(node.right, out);
}
```

Recursive postorder

```
void postorder(TreeNode node, List<Integer> out) {
    if (node == null) return;
    postorder(node.left, out);
    postorder(node.right, out);
    out.add(node.val);
}
```

Dry run

Traversal	Sample output
Preorder	1, 2, 4, 5, 7, 3, 6
Inorder	4, 2, 7, 5, 1, 3, 6
Postorder	4, 7, 5, 2, 6, 3, 1

Why recursion works: each call solves the same traversal problem on a smaller subtree. The base case stops at null. Every real node is processed exactly once, so time is O(n). Call-stack depth is O(h), which becomes O(n) for a skewed tree.

One method returning all three traversals in one DFS

```
class ThreeOrders {
    List<Integer> pre = new ArrayList<>();
    List<Integer> in = new ArrayList<>();
    List<Integer> post = new ArrayList<>();

    void dfs(TreeNode node) {
        if (node == null) return;
        pre.add(node.val);       // first visit
        dfs(node.left);
        in.add(node.val);        // between children
        dfs(node.right);
        post.add(node.val);      // after children
    }
}
```

This is useful for understanding traversal timing. It is not a replacement for the separate standard methods in every platform question; initialize/clear result lists before reusing an object.

5. Iterative DFS traversals

An explicit stack simulates the recursive call stack. Java's ArrayDeque is a good stack: push(), pop(), peek(). Do not insert null into ArrayDeque.

Iterative preorder — root, left, right

```
List<Integer> preorderIter(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;

    Deque<TreeNode> st = new ArrayDeque<>();
    st.push(root);
    while (!st.isEmpty()) {
        TreeNode cur = st.pop();
        ans.add(cur.val);
        if (cur.right != null) st.push(cur.right); // push right first
        if (cur.left != null) st.push(cur.left);   // left pops first
    }
    return ans;
}
```

Stack is LIFO. To process left before right, push right first. This small ordering detail is a frequent interview bug.

Iterative inorder — left, root, right

```
List<Integer> inorderIter(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) {
            st.push(cur);
            cur = cur.left;
        }
        cur = st.pop();
        ans.add(cur.val);
        cur = cur.right;
    }
    return ans;
}
```

The inner loop pushes the entire left chain. Once no left child remains, pop/process the next ancestor, then explore its right subtree.

Iterative postorder — two stacks

```
List<Integer> postorderTwoStacks(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    Deque<TreeNode> s1 = new ArrayDeque<>();
    Deque<TreeNode> s2 = new ArrayDeque<>();
    s1.push(root);

    while (!s1.isEmpty()) {
        TreeNode cur = s1.pop();
        s2.push(cur);
        if (cur.left != null) s1.push(cur.left);
        if (cur.right != null) s1.push(cur.right);
    }
    while (!s2.isEmpty()) ans.add(s2.pop());
    return ans;
}
```

First stack produces a root-left-right-like sequence in a controlled order; the second reverses it into left-right-root.

Iterative postorder — one stack

```
List<Integer> postorderOneStack(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root, lastVisited = null;

    while (cur != null || !st.isEmpty()) {
        if (cur != null) {
            st.push(cur);
            cur = cur.left;
        } else {
            TreeNode peek = st.peek();
            if (peek.right != null && lastVisited != peek.right) {
                cur = peek.right;
            } else {
                ans.add(peek.val);
                lastVisited = st.pop();
            }
        }
    }
    return ans;
}
```

The lastVisited pointer answers: has the right subtree already been completed? If not, traverse it; otherwise process the parent.

6. BFS — Level Order and common views

Breadth-First Search visits all nodes at distance 0, then distance 1, then distance 2, etc. A FIFO queue guarantees this order. Time O(n), auxiliary queue O(w), where w is the maximum number of nodes at any level.

Level order grouped by levels

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;

    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size(); // freeze this level's node count
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < size; i++) {
            TreeNode cur = q.poll();
            level.add(cur.val);
            if (cur.left != null) q.offer(cur.left);
            if (cur.right != null) q.offer(cur.right);
        }
        ans.add(level);
    }
    return ans;
}
```

Do not calculate q.size() in the loop condition. Children added during this iteration belong to the next level.

Reverse level order (bottom level first)

```
List<List<Integer>> reverseLevelOrder(TreeNode root) {
    LinkedList<List<Integer>> ans = new LinkedList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode cur = q.poll();
            level.add(cur.val);
            if (cur.left != null) q.offer(cur.left);
            if (cur.right != null) q.offer(cur.right);
        }
        ans.addFirst(level);
    }
    return ans;
}
```

Right and left side views

```
List<Integer> rightSideView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode cur = q.poll();
            if (i == size - 1) ans.add(cur.val);
            if (cur.left != null) q.offer(cur.left);
            if (cur.right != null) q.offer(cur.right);
        }
    }
    return ans;
}

List<Integer> leftSideView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode cur = q.poll();
            if (i == 0) ans.add(cur.val);
            if (cur.left != null) q.offer(cur.left);
            if (cur.right != null) q.offer(cur.right);
        }
    }
    return ans;
}
```

Right view captures the last BFS node at each level; left view captures the first. A DFS alternative can capture the first node seen at each depth by choosing the preferred side first.

7. Vertical, top, bottom and diagonal views

Assign each node a horizontal distance (HD): root=0, left child=HD−1, right child=HD+1. BFS plus a sorted map groups columns from left to right. Tie-breaking rules vary across platforms; read whether same-row nodes are ordered by encounter order or by value.

Vertical order traversal (BFS encounter order)

```
static class HDNode {
    TreeNode node; int hd;
    HDNode(TreeNode node, int hd) { this.node=node; this.hd=hd; }
}

List<List<Integer>> verticalOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;

    TreeMap<Integer, List<Integer>> map = new TreeMap<>();
    Queue<HDNode> q = new ArrayDeque<>();
    q.offer(new HDNode(root, 0));

    while (!q.isEmpty()) {
        HDNode item = q.poll();
        map.computeIfAbsent(item.hd, k -> new ArrayList<>())
           .add(item.node.val);
        if (item.node.left != null)
            q.offer(new HDNode(item.node.left, item.hd - 1));
        if (item.node.right != null)
            q.offer(new HDNode(item.node.right, item.hd + 1));
    }
    for (List<Integer> col : map.values()) ans.add(col);
    return ans;
}
```

Top view

For each horizontal distance, the first BFS node encountered is the shallowest visible node. Keep the first value for each HD, then output in sorted HD order.

```
List<Integer> topView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    TreeMap<Integer, Integer> first = new TreeMap<>();
    Queue<HDNode> q = new ArrayDeque<>();
    q.offer(new HDNode(root, 0));
    while (!q.isEmpty()) {
        HDNode x = q.poll();
        first.putIfAbsent(x.hd, x.node.val);
        if (x.node.left != null) q.offer(new HDNode(x.node.left, x.hd-1));
        if (x.node.right != null) q.offer(new HDNode(x.node.right, x.hd+1));
    }
    ans.addAll(first.values());
    return ans;
}
```

Bottom view

For each HD, overwrite the stored value whenever a node is visited in BFS. Later/deeper nodes replace earlier ones. If same-depth collisions exist, tie-breaking may need explicit row/column/value sorting.

```
List<Integer> bottomView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    TreeMap<Integer, Integer> last = new TreeMap<>();
    Queue<HDNode> q = new ArrayDeque<>();
    q.offer(new HDNode(root, 0));
    while (!q.isEmpty()) {
        HDNode x = q.poll();
        last.put(x.hd, x.node.val);
        if (x.node.left != null) q.offer(new HDNode(x.node.left, x.hd-1));
        if (x.node.right != null) q.offer(new HDNode(x.node.right, x.hd+1));
    }
    ans.addAll(last.values());
    return ans;
}
```

Diagonal traversal idea

A common convention: moving right stays on the same diagonal; moving left increases diagonal number. Use a queue of left children while repeatedly following right pointers. Since platforms differ in diagonal ordering, define the expected order before implementing.

8. Height, depth, size, leaves and width

Height / maximum depth

```
int height(TreeNode root) {
    if (root == null) return 0;
    int lh = height(root.left);
    int rh = height(root.right);
    return 1 + Math.max(lh, rh);
}
```

Recurrence: H(null)=0; H(node)=1+max(H(left),H(right)). Time O(n), stack O(h). If height is measured in edges, use H(null)=–1, so a leaf has height 0.

Depth of a value

```
int depthOf(TreeNode root, int target, int depth) {
    if (root == null) return -1;
    if (root.val == target) return depth;
    int left = depthOf(root.left, target, depth + 1);
    if (left != -1) return left;
    return depthOf(root.right, target, depth + 1);
}
// Call depthOf(root, target, 0)
```

If values can repeat, searching by value may find the wrong node. Use a TreeNode reference/identity when the problem refers to a particular node.

Count all nodes and leaf nodes

```
int countNodes(TreeNode root) {
    if (root == null) return 0;
    return 1 + countNodes(root.left) + countNodes(root.right);
}

int countLeaves(TreeNode root) {
    if (root == null) return 0;
    if (root.left == null && root.right == null) return 1;
    return countLeaves(root.left) + countLeaves(root.right);
}
```

Diameter — longest path between any two nodes

Diameter is measured in edges. At a node, the longest path passing through that node has leftHeight + rightHeight edges (when height uses node counts). The global answer is the maximum over all nodes. The path does not have to pass through the root.

```
class Diameter {
    int best = 0;

    int heightAndUpdate(TreeNode node) {
        if (node == null) return 0;
        int left = heightAndUpdate(node.left);
        int right = heightAndUpdate(node.right);
        best = Math.max(best, left + right);
        return 1 + Math.max(left, right);
    }

    int diameterOfBinaryTree(TreeNode root) {
        best = 0;
        heightAndUpdate(root);
        return best;
    }
}
```

Why O(n): each node returns its height once and updates the global answer once. The naive approach calls a height function for every node, which can be O(n²) on a chain.

Maximum width

Two definitions appear in interviews: actual node count on a level, or positional width including null gaps between leftmost/rightmost nodes. For positional width, assign complete-tree indices: left=2i+1, right=2i+2. Normalize by subtracting the first index at each level to reduce overflow.

```
long maxWidthWithGaps(TreeNode root) {
    if (root == null) return 0;
    class Item {
        TreeNode node; long idx;
        Item(TreeNode n, long i) { node=n; idx=i; }
    }
    Queue<Item> q = new ArrayDeque<>();
    q.offer(new Item(root, 0));
    long best = 0;
    while (!q.isEmpty()) {
        int size = q.size();
        long base = q.peek().idx, first = 0, last = 0;
        for (int i=0; i<size; i++) {
            Item x = q.poll();
            long id = x.idx - base;
            if (i == 0) first = id;
            if (i == size-1) last = id;
            if (x.node.left != null) q.offer(new Item(x.node.left, 2*id+1));
            if (x.node.right != null) q.offer(new Item(x.node.right, 2*id+2));
        }
```

```
            best = Math.max(best, last-first+1);
        }
        return best;
    }
```

9. Balanced Binary Tree

A common height-balanced definition says that at every node, the height difference between left and right subtree is at most 1. Balance factor = height(left) – height(right). A node is balanced if factor is –1, 0 or +1.

Naive solution — why it is slow

```
boolean balancedNaive(TreeNode root) {
    if (root == null) return true;
    int lh = height(root.left);
    int rh = height(root.right);
    return Math.abs(lh-rh) <= 1
        && balancedNaive(root.left)
        && balancedNaive(root.right);
}
```

At each node this recomputes heights of descendants. In a skewed tree the total work can be $O(n^2)$.

Optimized one-pass postorder solution

```
class Balanced {
    int check(TreeNode node) {
        if (node == null) return 0;

        int lh = check(node.left);
        if (lh == -1) return -1;

        int rh = check(node.right);
        if (rh == -1) return -1;

        if (Math.abs(lh-rh) > 1) return -1;
        return 1 + Math.max(lh, rh);
    }

    boolean isBalanced(TreeNode root) {
        return check(root) != -1;
    }
}
```

Return either subtree height or sentinel –1 for unbalanced. A parent immediately propagates –1. Each node is visited once: O(n) time and O(h) stack.

AVL tree vs checking balance

A balance-check function only answers true/false; it does not rebalance. AVL is a self-balancing BST and maintains |balance factor|≤1 after updates by rotations.

Imbalance	Shape	Fix
LL	New key inserted in left subtree of left child	Right rotation at unbalanced node
RR	New key inserted in right subtree of right child	Left rotation at unbalanced node
LR	Left child is right-heavy	Left rotate left child, then right rotate node
RL	Right child is left-heavy	Right rotate right child, then left rotate node

Interview variations

- Check whether a tree is height-balanced.
- Find the height and balance factor of every node.
- Explain why a single postorder traversal is enough.
- Compare AVL, Red-Black Tree and an ordinary BST at a high level.

10. Lowest Common Ancestor (LCA)

LCA(p,q) is the deepest node that is an ancestor of both p and q. A node is allowed to be its own ancestor. Clarify whether both target nodes are guaranteed to exist.

General Binary Tree — recursive solution

```
TreeNode lowestCommonAncestor(TreeNode root,
                              TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

Logic: if one target is found in each side, current root is the split point and therefore LCA. If both are in one side, propagate that side's result. Time O(n), stack O(h). This standard version assumes both nodes exist; otherwise it may return the one that exists.

LCA when a target may be absent

Return both a candidate and a count/bitmask of targets found. Accept the candidate only if both targets were found. One simple alternative is two separate contains() searches after finding a candidate, still O(n) asymptotically.

```
boolean contains(TreeNode root, TreeNode target) {
    if (root == null) return false;
    if (root == target) return true;
    return contains(root.left, target) || contains(root.right, target);
}

TreeNode lcaOnlyIfBothExist(TreeNode root, TreeNode p, TreeNode q) {
    if (!contains(root, p) || !contains(root, q)) return null;
    return lowestCommonAncestor(root, p, q);
}
```

LCA in a BST

```
TreeNode lcaBST(TreeNode root, TreeNode p, TreeNode q) {
    TreeNode cur = root;
    while (cur != null) {
        if (p.val < cur.val && q.val < cur.val) {
            cur = cur.left;
        } else if (p.val > cur.val && q.val > cur.val) {
            cur = cur.right;
        } else {
            return cur; // split point or one target is current
        }
    }
    return null;
}
```

BST LCA is O(h) time and O(1) iterative space. General-tree LCA cannot use value ordering; it must explore the structure.

LCA dry-run reasoning

- If p and q are on opposite sides of node X, X is their LCA.
- If p is an ancestor of q, p is the LCA.
- If both are in left subtree, search left; if both are in right subtree, search right.
- With repeated values, compare node references rather than values if the problem identifies particular nodes.

11. Path Sum problems — understand path definition

A path is a sequence of connected nodes. Interview variants differ in allowed start/end points. Root-to-leaf is not the same as any-node-to-any-node.

Path Sum I — root to leaf

```
boolean hasPathSum(TreeNode root, long target) {
    if (root == null) return false;
    target -= root.val;

    if (root.left == null && root.right == null)
        return target == 0;

    return hasPathSum(root.left, target)
        || hasPathSum(root.right, target);
}
```

A leaf check is essential. Reaching a node with remaining sum zero is not enough if that node has children; the required path must end at a leaf.

Return every root-to-leaf path with target sum

```
void pathSumAll(TreeNode node, long remaining,
                List<Integer> path,
                List<List<Integer>> ans) {
    if (node == null) return;

    path.add(node.val);
    remaining -= node.val;

    if (node.left == null && node.right == null
            && remaining == 0) {
        ans.add(new ArrayList<>(path)); // save a copy
    } else {
        pathSumAll(node.left, remaining, path, ans);
        pathSumAll(node.right, remaining, path, ans);
    }

    path.remove(path.size()-1); // backtrack
}
```

Backtracking rule: add current node before exploring children; remove it after both children. Copy the list when saving a result, otherwise all answers may point to the same mutable list.

Maximum Path Sum — any node to any node

A maximum path may begin and end anywhere, but cannot branch. At a node, the best path passing through it can include left gain + node + right gain. The value returned to parent can extend in only one direction, so return node + max(left,right). Negative child gains are ignored.

```
class MaximumPathSum {
    int best;

    int gain(TreeNode node) {
        if (node == null) return 0;

        int left = Math.max(0, gain(node.left));
        int right = Math.max(0, gain(node.right));

        best = Math.max(best, node.val + left + right);
        return node.val + Math.max(left, right);
    }

    int maxPathSum(TreeNode root) {
        best = Integer.MIN_VALUE;
        gain(root);
        return best;
    }
}
```

Do not initialize best to zero: an all-negative tree must return its largest (least negative) node. Use long if constraints permit sums beyond int range.

Other path patterns

• Root-to-leaf path listing: DFS + mutable path + backtracking.

• Count paths whose sum equals target anywhere: prefix-sum frequency map; maintain prefix sum while DFS enters/exits nodes.

• Maximum root-to-leaf sum: return node value plus max child result; handle missing children carefully.

• Path between two nodes: find LCA, then build paths from LCA to each target.

12. Boundary traversal and Zigzag

Anti-clockwise boundary order

Standard order: (1) root once, (2) left boundary excluding leaves, preferring left child then right fallback, (3) all leaves left-to-right, (4) right boundary excluding leaves in reverse, preferring right child then left fallback. If root is a leaf, output it once.

```
boolean isLeaf(TreeNode n) {
    return n != null && n.left == null && n.right == null;
}

void addLeftBoundary(TreeNode root, List<Integer> ans) {
    TreeNode cur = root.left;
    while (cur != null) {
        if (!isLeaf(cur)) ans.add(cur.val);
        cur = (cur.left != null) ? cur.left : cur.right;
    }
}

void addLeaves(TreeNode node, List<Integer> ans) {
    if (node == null) return;
    if (isLeaf(node)) { ans.add(node.val); return; }
    addLeaves(node.left, ans);
    addLeaves(node.right, ans);
}

void addRightBoundary(TreeNode root, List<Integer> ans) {
    TreeNode cur = root.right;
    List<Integer> temp = new ArrayList<>();
    while (cur != null) {
        if (!isLeaf(cur)) temp.add(cur.val);
        cur = (cur.right != null) ? cur.right : cur.left;
    }
    for (int i=temp.size()-1; i>=0; i--) ans.add(temp.get(i));
}

List<Integer> boundaryTraversal(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    if (!isLeaf(root)) ans.add(root.val);
    addLeftBoundary(root, ans);
    addLeaves(root, ans);
    addRightBoundary(root, ans);
    return ans;
}
```

Time O(n), extra space O(h) for leaf recursion plus O(h) temporary right boundary. Boundary definitions can vary slightly; match the platform's required clockwise/anti-clockwise order.

Zigzag / spiral level order

BFS still discovers children left-to-right. Only the output index changes: even depth uses i; odd depth uses size–1–i.

```
List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;

    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    boolean leftToRight = true;

    while (!q.isEmpty()) {
        int size = q.size();
        Integer[] level = new Integer[size];

        for (int i=0; i<size; i++) {
            TreeNode cur = q.poll();
            int pos = leftToRight ? i : size-1-i;
            level[pos] = cur.val;

            if (cur.left != null) q.offer(cur.left);
            if (cur.right != null) q.offer(cur.right);
        }
        ans.add(Arrays.asList(level));
        leftToRight = !leftToRight;
    }
    return ans;
}
```

Zigzag dry run

Level	Normal BFS	Zigzag output
0	1	1
1	2, 3	3, 2
2	4, 5, 6	4, 5, 6
3	…	reverse order at this level

13. Serialize and Deserialize

Serialization converts a tree into a linear representation; deserialization reconstructs the same structure. Values alone are insufficient because they do not encode missing left/right children. Null markers preserve shape.

Preorder + null marker approach

Example for a root 1 with left child 2 and no right child: 1,2,#,#,#. Each non-null node contributes its value followed by encodings of left and right. Each null child contributes #.

```
class Codec {
    private static final String SEP = ",";
    private static final String NIL = "#";

    String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        write(root, sb);
        return sb.toString();
    }

    private void write(TreeNode node, StringBuilder sb) {
        if (node == null) {
            sb.append(NIL).append(SEP);
            return;
        }
        sb.append(node.val).append(SEP);
        write(node.left, sb);
        write(node.right, sb);
    }

    TreeNode deserialize(String data) {
        if (data == null || data.isEmpty())
            throw new IllegalArgumentException("Empty encoding");
        String[] tokens = data.split(",");
        int[] index = {0};
        TreeNode root = read(tokens, index);
        if (index[0] != tokens.length)
            throw new IllegalArgumentException("Extra tokens");
        return root;
    }

    private TreeNode read(String[] a, int[] index) {
        if (index[0] >= a.length)
            throw new IllegalArgumentException("Truncated encoding");

        String token = a[index[0]++];
        if (token.equals(NIL)) return null;

        TreeNode node = new TreeNode(Integer.parseInt(token));
        node.left = read(a, index);
        node.right = read(a, index);
        return node;
    }
}
```

Time O(n) to serialize and deserialize; encoded output O(n). Recursive call stack O(h). For very deep trees, iterative serialization/deserialization may avoid stack overflow. This format assumes integer values and comma is reserved as delimiter.

Why null markers matter

- Without null markers, trees with a left child and a right child can produce the same preorder values.
- Every node has exactly two child slots; encoding each missing slot makes the structure unambiguous.
- An empty tree serializes to #.
- Deserialization consumes tokens in exactly the same preorder order used by serialization.

Level-order serialization concept

Use a queue, output each node value, and output null placeholders for absent children. A decoder reads tokens and attaches left/right children in queue order. If trailing null markers are trimmed, define that rule in both encoder and decoder.

14. Construct tree from traversal arrays

For distinct values, inorder paired with preorder or postorder uniquely identifies the tree. Preorder + postorder alone is generally ambiguous because a one-child node could place that child on either side.

Preorder + Inorder

Preorder[0] is root. Locate it at index m in inorder. Left inorder segment is left subtree; right segment is right subtree. Preorder then contains the corresponding left-subtree values followed by right-subtree values.

```
class BuildPreIn {
    int preIndex;
    Map<Integer, Integer> position;

    TreeNode buildTree(int[] preorder, int[] inorder) {
        preIndex = 0;
        position = new HashMap<>();
        for (int i=0; i<inorder.length; i++)
            position.put(inorder[i], i);
        return build(preorder, 0, inorder.length-1);
    }

    TreeNode build(int[] pre, int lo, int hi) {
        if (lo > hi) return null;
        int value = pre[preIndex++];
        TreeNode root = new TreeNode(value);
        int mid = position.get(value);
        root.left = build(pre, lo, mid-1);
        root.right = build(pre, mid+1, hi);
        return root;
    }
}
```

HashMap gives O(1) average index lookup. Total time O(n), map O(n), recursion O(h). Assumes valid traversals and unique values. With duplicates, value-to-index lookup is ambiguous and needs a different strategy.

Inorder + Postorder

Postorder's last element is root. Since the pointer moves backward, build right subtree first, then left.

```
class BuildInPost {
    int postIndex;
    Map<Integer, Integer> position;

    TreeNode buildTree(int[] inorder, int[] postorder) {
        postIndex = postorder.length-1;
        position = new HashMap<>();
        for (int i=0; i<inorder.length; i++)
            position.put(inorder[i], i);
        return build(postorder, 0, inorder.length-1);
    }

    TreeNode build(int[] post, int lo, int hi) {
        if (lo > hi) return null;
        int value = post[postIndex--];
        TreeNode root = new TreeNode(value);
        int mid = position.get(value);
        root.right = build(post, mid+1, hi);
        root.left = build(post, lo, mid-1);
        return root;
    }
}
```

Why preorder + postorder alone is ambiguous

For preorder [A,B] and postorder [B,A], B may be A's left child or A's right child. Both shapes produce the same traversals. If the tree is guaranteed full (each internal node has exactly two children) and values are distinct, reconstruction is possible.

Build BST from preorder using bounds

```
class BSTFromPreorder {
    int index;

    TreeNode buildBST(int[] preorder) {
        index = 0;
        return build(preorder, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    TreeNode build(int[] a, long low, long high) {
        if (index == a.length) return null;
        int value = a[index];
        if (value <= low || value >= high) return null;

        index++;
        TreeNode node = new TreeNode(value);
        node.left = build(a, low, value);
        node.right = build(a, value, high);
        return node;
    }
}
```

Each value is consumed once, so O(n) time and O(h) recursion. The bounds enforce the BST invariant without scanning for split points.

15. Morris Traversal — O(1) auxiliary space

Morris traversal avoids recursion and explicit stack by temporarily changing null right pointers into threads. It runs in O(n) time and O(1) auxiliary space, excluding the output list. The original tree links must be restored.

Morris Inorder

At current node: (1) if left is null, visit current and move right; (2) otherwise find inorder predecessor, the rightmost node in current.left. If predecessor.right is null, connect it to current and move left. If it already points to current, remove the thread, visit current and move right.

```
List<Integer> morrisInorder(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    TreeNode cur = root;

    while (cur != null) {
        if (cur.left == null) {
            ans.add(cur.val);
            cur = cur.right;
        } else {
            TreeNode pred = cur.left;
            while (pred.right != null && pred.right != cur)
                pred = pred.right;

            if (pred.right == null) {
                pred.right = cur;  // temporary thread
                cur = cur.left;
            } else {
                pred.right = null; // restore original link
                ans.add(cur.val);
                cur = cur.right;
            }
        }
    }
    return ans;
}
```

Morris Preorder

Same predecessor/thread logic. Visit current at the moment the thread is first created, because preorder visits root before left subtree. On the second encounter, remove thread and do not visit again.

```
List<Integer> morrisPreorder(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    TreeNode cur = root;

    while (cur != null) {
        if (cur.left == null) {
            ans.add(cur.val);
            cur = cur.right;
        } else {
            TreeNode pred = cur.left;
            while (pred.right != null && pred.right != cur)
                pred = pred.right;

            if (pred.right == null) {
                ans.add(cur.val);  // preorder visit
                pred.right = cur;
                cur = cur.left;
            } else {
                pred.right = null;
                cur = cur.right;
            }
        }
    }
    return ans;
}
```

Morris traversal pitfalls

• Do not forget to restore pred.right = null when returning through a thread.

• Do not visit a node twice: inorder visits on second encounter; preorder visits on first.

• The tree is temporarily mutated. Do not use when concurrent readers require a stable tree.

• If execution is interrupted before all threads are removed, the structure may be left modified.

• Output list itself is O(n); O(1) claim refers to auxiliary traversal space excluding output.

16. Structural binary-tree algorithms

Invert / mirror a binary tree

```
TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;
    invertTree(root.left);
    invertTree(root.right);
    return root;
}
```

Swap children at every node. Time O(n), stack O(h).

Same tree

```
boolean isSameTree(TreeNode a, TreeNode b) {
    if (a == null || b == null) return a == b;
    return a.val == b.val
        && isSameTree(a.left, b.left)
        && isSameTree(a.right, b.right);
}
```

Symmetric tree

```
boolean isSymmetric(TreeNode root) {
    return root == null || mirror(root.left, root.right);
}
boolean mirror(TreeNode a, TreeNode b) {
    if (a == null || b == null) return a == b;
    return a.val == b.val
        && mirror(a.left, b.right)
        && mirror(a.right, b.left);
}
```

Subtree check

```
boolean isSubtree(TreeNode root, TreeNode sub) {
    if (sub == null) return true;
    if (root == null) return false;
    if (isSameTree(root, sub)) return true;
    return isSubtree(root.left, sub) || isSubtree(root.right, sub);
}
```

This straightforward solution can be O(nm) in worst case for n nodes in main tree and m in candidate subtree. For large constraints, use serialization with careful null markers or tree hashing.

Flatten binary tree to linked list in preorder

After flattening, every left pointer is null and right pointers follow preorder. One reverse-preorder recursion carries the previously processed node.

```
class Flatten {
    TreeNode previous = null;

    void flatten(TreeNode root) {
        if (root == null) return;
        flatten(root.right);
        flatten(root.left);
        root.right = previous;
        root.left = null;
        previous = root;
    }
}
```

Time O(n), stack O(h). Iterative alternative: at each node with a left child, find rightmost node of left subtree, connect it to current.right, move left subtree to right, and clear left.

17. Complexity master table

Algorithm	Time	Auxiliary space	Reason / note
Recursive DFS traversals	O(n)	O(h)	Each node once; call stack height
Iterative DFS	O(n)	O(h) to O(n)	Explicit stack
Level order / views	O(n)	O(w)	Queue stores a level
Height / count / leaves	O(n)	O(h)	Postorder recursion
Diameter optimized	O(n)	O(h)	Height returned once per node
Balanced optimized	O(n)	O(h)	Height + failure sentinel
LCA general tree	O(n)	O(h)	May inspect all nodes
LCA BST	O(h)	O(1) iterative	Ordering discards a subtree
Path sum	O(n)	O(h)	DFS
Boundary traversal	O(n)	O(h)	Leaves + boundary
Serialize / deserialize	O(n)	O(n) output	Every node/null slot encoded
Build from traversals + map	O(n)	O(n)	Hash lookup per root
Morris traversal	O(n)	O(1)*	Temporary threads; output excluded
BST search/insert/delete	O(h)	O(1) iterative	h may equal n

*Morris O(1) refers only to auxiliary traversal storage, not the returned list. If output is streamed directly to a consumer, output storage may be avoided.

n, h and w — know the variables

• n = total number of nodes.

• h = tree height. Balanced tree often h=O(log n); skewed tree h=O(n).

• w = maximum number of nodes on a level; queue space for BFS is O(w), worst O(n).

• Do not automatically write O(log n) for BST. Ordinary BST can become skewed.

How to derive complexity

Count how many times each node is visited for time. For recursion, maximum simultaneous calls equal height h. For BFS, queue can contain many nodes from one level, bounded by width w. For reconstruction with a HashMap, map uses O(n) space and gives average O(1) index lookup.

18. Common mistakes and debugging checklist

Mistake	Correct reasoning
Height vs depth confusion	Depth is root-to-node; height is node-to-leaf. State units.
Diameter = height	Diameter is longest path between any two nodes; it may not pass through root.
Only compare BST parent and child	Validate using ancestor bounds or inorder ordering.
Push left before right in preorder stack	Push right first because stack is LIFO.
Use q.size() dynamically inside BFS loop	Capture current level size before processing.
Treat reaching sum 0 as root-to-leaf success	Must also be at a leaf.
Forget to copy a path result	Store new ArrayList<>(path), then backtrack.
Initialize max path sum to 0	All-negative trees require Integer.MIN_VALUE.
Assume preorder+postorder always unique	One-child orientation is ambiguous without extra constraints.
Ignore duplicate traversal values	A simple value→index map requires distinct values.
Forget Morris thread restoration	Remove temporary predecessor.right links.
Claim BST operations always O(log n)	They are O(h); skewed BST gives O(n).
Put null in ArrayDeque	ArrayDeque disallows null; only enqueue non-null nodes.

Must-test edge cases for every solution

• Empty tree: root == null.

• Single node: node is simultaneously root and leaf; diameter in edges is 0.

• Only-left chain and only-right chain: recursion depth reaches n.

• Perfect/complete tree: checks level boundaries and traversal order.

• Negative values: path sum and maximum path sum.

• Integer extremes: BST validation bounds; use long where necessary.

• Missing LCA target: confirm expected behavior.

• Duplicate values: identify whether node identity or key equality is used.

19. Interview problem bank — foundation to advanced

#	Problem / task	Main pattern
1	Find height / maximum depth	Postorder DFS
2	Find minimum depth to a leaf	BFS or DFS with leaf check
3	Count total nodes and leaf nodes	DFS recurrence
4	Preorder / inorder / postorder recursively	DFS
5	All traversals iteratively	Stack simulation
6	Level order traversal	BFS queue
7	Reverse level order	BFS + addFirst
8	Zigzag level order	BFS + reverse index
9	Left and right side views	BFS level endpoints / DFS
10	Top and bottom views	HD + BFS + ordered map
11	Vertical order traversal	HD grouping
12	Boundary traversal	Boundary components + leaves
13	Diameter of binary tree	Postorder + global max
14	Check balanced binary tree	Postorder + sentinel
15	Lowest common ancestor	Recursive split point
16	Path Sum I / II	DFS + backtracking
17	Maximum path sum	Tree DP / one-side gain
18	Serialize and deserialize	Traversal + null markers
19	Build from preorder + inorder	Index map + recursion
20	Build from inorder + postorder	Reverse pointer + recursion
21	Validate BST	Bounds
22	Kth smallest / largest in BST	Inorder / reverse inorder
23	BST successor / predecessor	Candidate ancestor
24	Delete node in BST	Successor replacement
25	Range sum BST	Pruning
26	Invert / mirror tree	DFS swap
27	Symmetric tree	Mirror recursion
28	Same tree / subtree	Structural comparison
29	Flatten tree to linked list	Reverse preorder / pointer rewiring
30	Morris inorder / preorder	Threaded traversal
31	Construct BST from preorder	Bounds + moving index
32	Maximum width including gaps	Indexed BFS

Practice protocol

For each problem, do not jump straight to code. Write: (1) input/output contract, (2) base case, (3) what the recursive call returns, (4) how left/right answers combine, (5) complexity, (6) edge cases. For iterative problems, state what the stack/queue represents.

Suggested progression

• Beginner: #1–#9 and #21–#22.

• Intermediate: #10–#20, #23–#29.

• Advanced: #30–#32, then solve variations without memorizing code.

20. Final formula sheet and quick revision

Concept	Formula / rule
Edges in tree	n–1
Max nodes at level d (root level 0)	2^d
Max nodes in h levels	2^h–1
Minimum height in levels for n nodes	ceil(log2(n+1))
Maximum height in levels	n (skewed tree)
Full binary tree	Every node has 0 or 2 children; leaves = internal + 1
0-based complete-tree array	left=2i+1, right=2i+2, parent=(i–1)/2
Height in nodes	H(null)=0; H(node)=1+max(H(left),H(right))
Diameter in edges	max over nodes of leftHeight + rightHeight
Balance condition	abs(leftHeight–rightHeight) ≤ 1 at every node
BST invariant	All left-subtree keys < node < all right-subtree keys
Preorder	Root, Left, Right
Inorder	Left, Root, Right
Postorder	Left, Right, Root
BFS	Queue; process captured level size
Morris	Temporary threads; O(1) auxiliary space

Explain these aloud — final self-test

• Why does a tree with n nodes have n–1 edges?

• Why does inorder traversal of a BST produce sorted values?

• Why is iterative preorder pushing right before left?

• Why is optimized diameter O(n) but naive diameter can be $O(n^2)$?

• Why does balanced-tree checking use a sentinel?

• Why is LCA the node where recursive results come from both sides?

• Why does maximum path sum ignore negative child gains?

• Why do serialization formats need null markers?

• Why does reconstruction from inorder+preorder work for distinct values?

• Why is preorder+postorder ambiguous in a general binary tree?

• How does Morris traversal visit nodes without a stack?

Interview-ready answer structure: Clarify the definition → state the key invariant → explain the algorithm → dry-run a non-trivial tree → give time and auxiliary-space complexity → discuss edge cases. A solution is not complete until you can explain why it works.